

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..7

SETUP

 Copy as Markdown

Set up Google sign-in

Sign in with Google requires a Google Cloud project. You also need platform-specific OAuth credentials depending on which platforms you target.

Prerequisites

Before following this guide, make sure you have:

- A Google account with access to [Google Cloud Console](#).
- A running Serverpod project (server, client, and Flutter app packages from `serverpod create`).
- The Serverpod auth module installed and configured per the [authentication setup](#). If your project was generated with an older Serverpod version, follow that guide first to add `serverpod_auth_idp_server` and `serverpod_auth_idp_flutter` and to configure `pod.initializeAuthServices()` before continuing.

Get your Google credentials

All platforms require a Web application OAuth client (used by the server). iOS and Android additionally require their own platform-specific OAuth clients.

Create a Google Cloud project

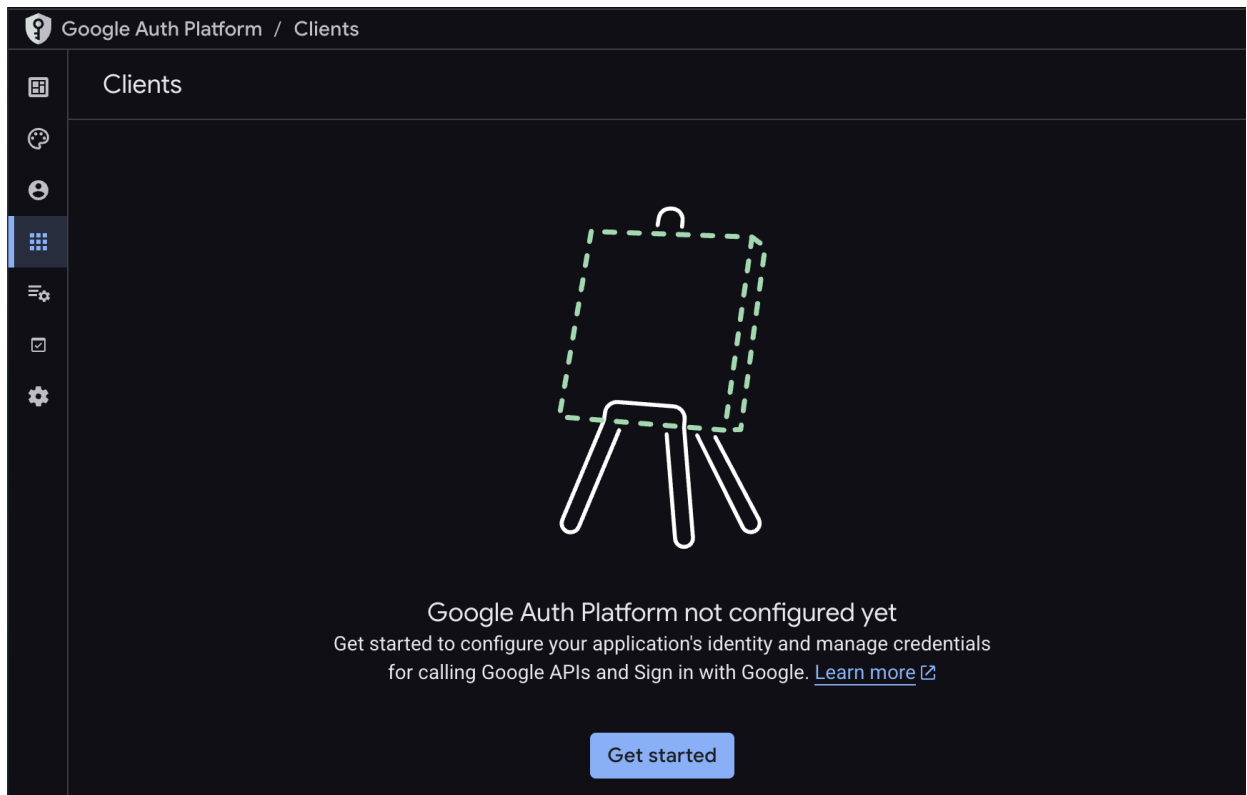
1. Go to [Create a project](#).
2. Enter a **Project name** (for example, `My Serverpod App`) and click **Create**.

Configure Google Auth Platform

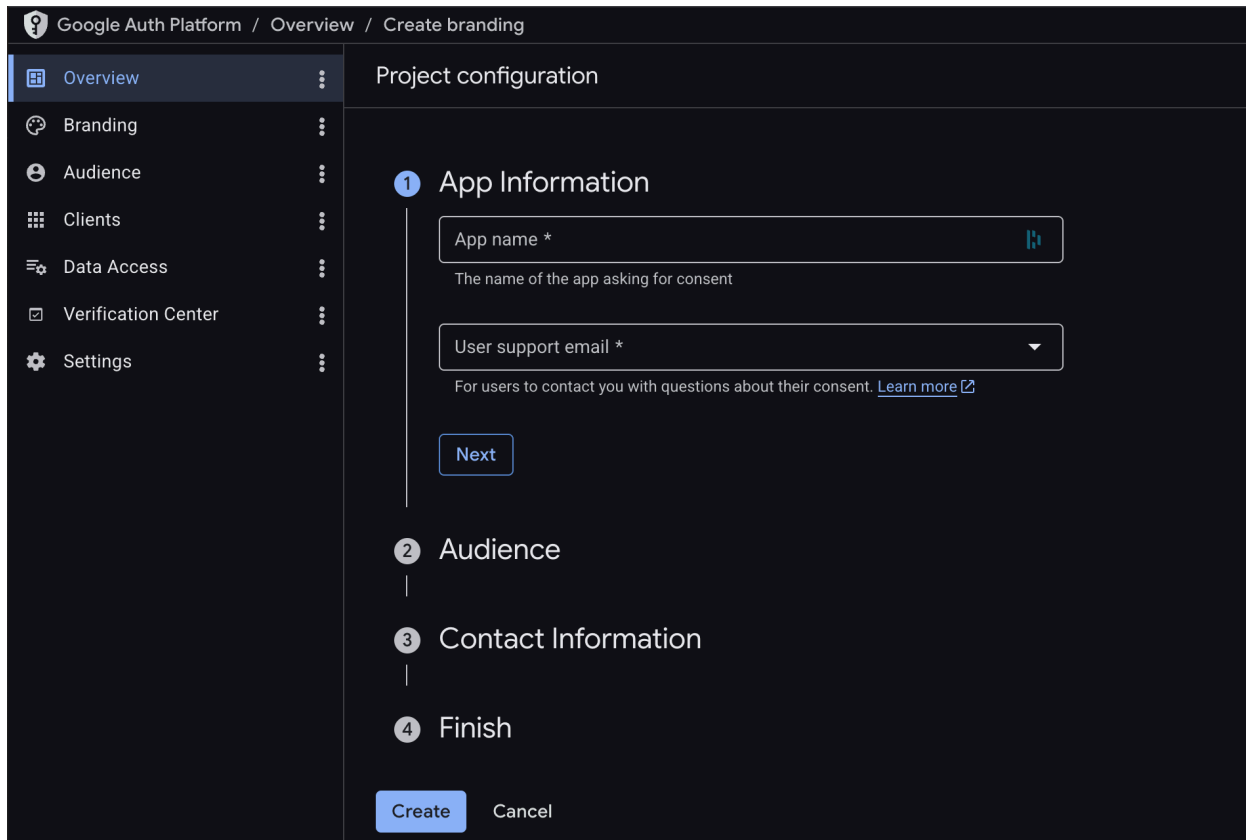
1. Navigate to the [Google Auth Platform overview](#) and click **Get started** if you haven't enabled



Ask AI



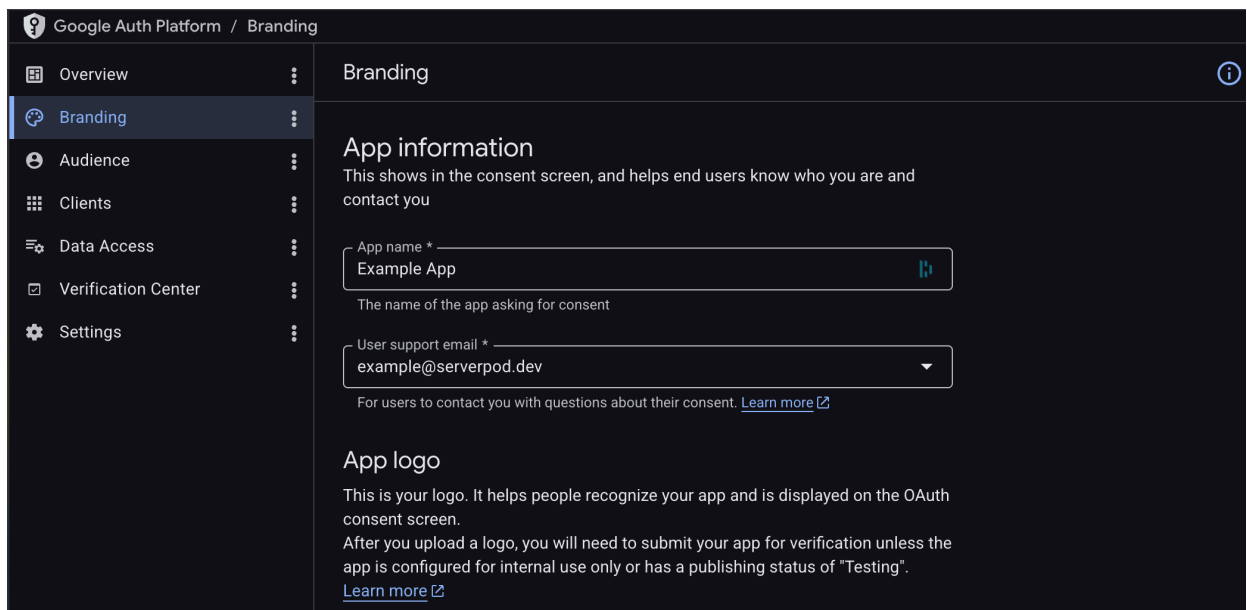
2. **Project configuration:** Complete the setup wizard by filling in the required fields across each step (App Information, Audience, Contact Information) and click **Create**.



3. **Branding:** After completing the wizard, navigate to the [Branding](#) page from the sidebar. Fill in the remaining fields: app logo, app homepage link, privacy policy link, terms of service link, developer contact email, and **authorized domains**. These details appear on the OAuth consent screen shown to users during sign-in.

Add the **root domain** you will deploy under to **Authorized domains**. Google stores only the top private domain, so a single root entry covers every subdomain you deploy under it.

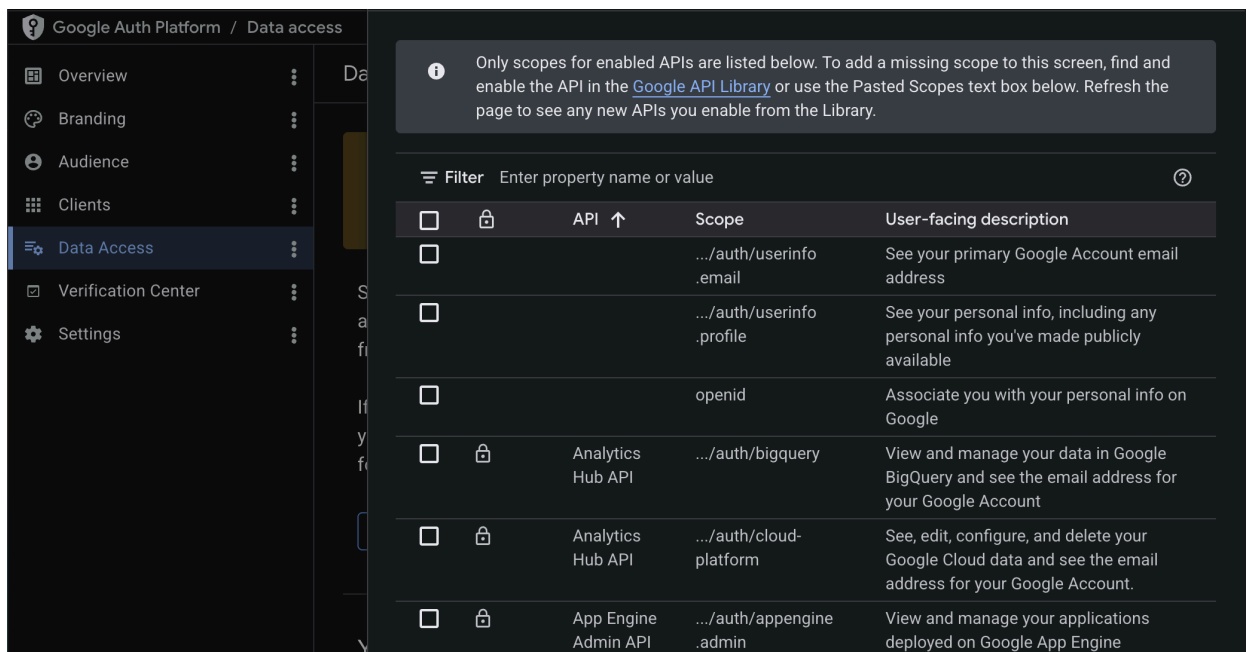
If you deploy on Serverpod Cloud, add `serverpod.space`. It is already verified by Serverpod, so you only need to add it here, no DNS verification is required on your end. For custom domains, see [Verify your authorized domain](#).



The screenshot shows the Google Auth Platform interface with the 'Branding' tab selected in the sidebar. The main content area is titled 'Branding' and contains the following sections:

- App information**: This section explains that the information shown here appears on the consent screen. It includes two input fields:
 - App name ***: A text input field containing 'Example App'.
 - User support email ***: A dropdown menu showing 'example@serverpod.dev'.
- App logo**: This section explains that the logo is used for app recognition and is displayed on the OAuth consent screen. It includes a note about the need for verification after uploading a logo and a 'Learn more' link.

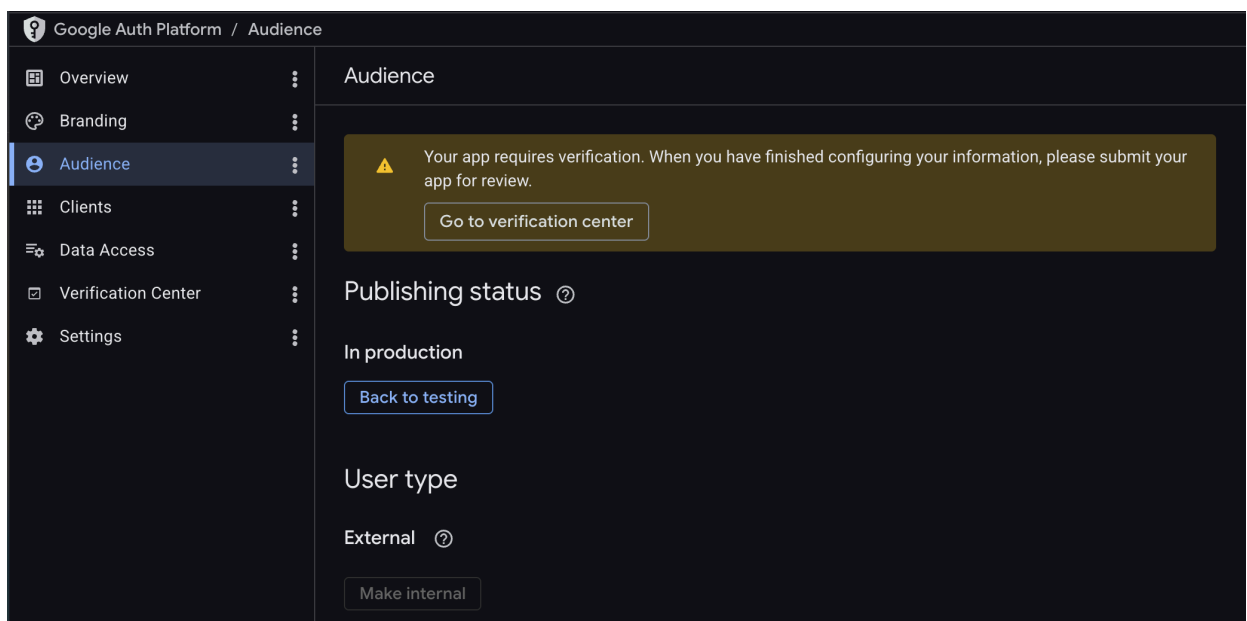
4. **Data access:** Navigate to the [Data Access](#) page and add the required scopes: `.../auth/userinfo.email` and `.../auth/userinfo.profile`.



TIP

If you need access to additional Google APIs (for example, Calendar or Drive), you can add more scopes here. See [Accessing Google APIs](#) for details on requesting additional scopes and using them with the `getExtraGoogleInfoCallback` on the server.

5. **Audience:** Navigate to the [Audience](#) page. While in development, the app is in **Testing** mode, which means only users you explicitly add as test users can sign in (up to 100). Add your email as a test user so you can test the integration.



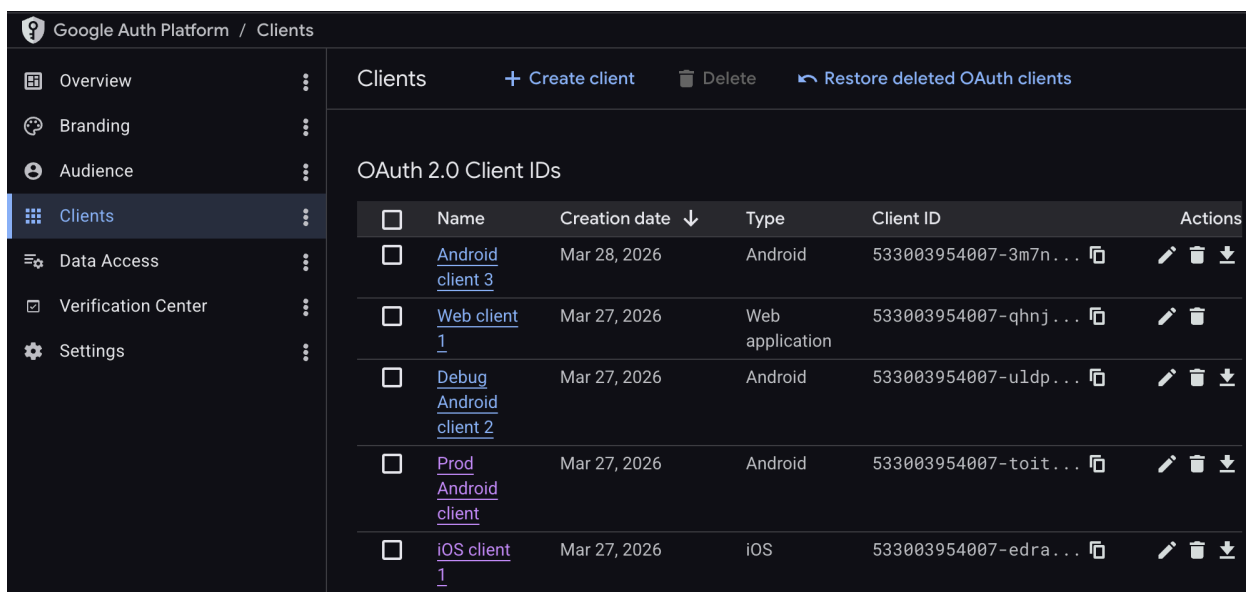
TIP

Leave the app in **Testing** mode for now. You can [publish it](#) after verifying that sign-in works end to end.

Create the server OAuth client (Web application)

All platforms (iOS, Android, and Web) require a **Web application** OAuth client for the server. This is the only client type that provides a **client secret**, which Serverpod needs to verify sign-in tokens on the server side.

1. In the Google Auth Platform, navigate to **Clients** and click **Create Client**.
2. Select **Web application** as the application type.
3. Leave **Authorized JavaScript origins** and **Authorized redirect URIs** empty for now. [Web setup](#) below fills them in if you target web.



The screenshot shows the Google Auth Platform interface. On the left is a sidebar with navigation links: Overview, Branding, Audience, Clients (selected), Data Access, Verification Center, and Settings. The main content area is titled 'Clients' and includes a '+ Create client' button, a 'Delete' button, and a 'Restore deleted OAuth clients' link. Below this is a table of 'OAuth 2.0 Client IDs'.

☐	Name	Creation date ↓	Type	Client ID	Actions
☐	Android client 3	Mar 28, 2026	Android	533003954007-3m7n...	
☐	Web client 1	Mar 27, 2026	Web application	533003954007-qhnj...	
☐	Debug Android client 2	Mar 27, 2026	Android	533003954007-uld...	
☐	Prod Android client	Mar 27, 2026	Android	533003954007-toit...	
☐	iOS client 1	Mar 27, 2026	iOS	533003954007-edra...	

4. Click **Create**.
5. Copy the **Client ID** and **Client secret** shown on screen. You will need both in the next step.

Store your credentials

Your server's `config/passwords.yaml` already has `development:`, `staging:`, and `production:` sections from the project template. Add the `googleClientSecret` key to the `development:` section using the client ID and client secret you copied:

```
development:
  # ... existing keys (database, redis, serviceSecret, etc.) ...
  googleClientSecret: |
    {
      "web": {
        "client_id": "your-client-id.apps.googleusercontent.com",
```

```

    "client_secret": "your-client-secret",
    "redirect_uris": []
  }
}

```

Replace `your-client-id` and `your-client-secret` with the values from the Google Auth Platform. If you target web, [Web setup](#) below adds the callback URL to `redirect_uris`.

For production, add the same `googleClientSecret` entry to the `production:` section of `passwords.yaml` (with your production redirect URI), or set the `SERVERPOD_PASSWORD_googleClientSecret` environment variable on your production server.

NOTE

Carefully maintain correct indentation for YAML block scalars. The `googleClientSecret` block uses a `|`. Any indentation error makes the JSON fail to parse, and the server throws at startup when `GoogleIdpConfigFromPasswords()` loads the secret.

Server-side configuration

Add the Google identity provider

Your server's `server.dart` file (for example, `my_project_server/lib/server.dart`) should already contain a `pod.initializeAuthServices()` call if your project was created with the Serverpod project template (`serverpod create`). If it's not there, see [Setup](#) first to configure the auth module and JWT settings.

Add the Google import and `GoogleIdpConfigFromPasswords()` to the existing `identityProviderBuilders` list:

```
import 'package:serverpod_auth_idp_server/providers/google.dart';
```

```

pod.initializeAuthServices(
  tokenManagerBuilders: [
    JwtConfigFromPasswords(),
  ],
  identityProviderBuilders: [
    // ... any existing providers (e.g., EmailIdpConfigFromPasswords) ...
    GoogleIdpConfigFromPasswords(),
  ],
);

```

The `GoogleIdpConfigFromPasswords()` constructor automatically loads the client secret from the `googleClientSecret` key in `config/passwords.yaml` (or the `SERVERPOD_PASSWORD_googleClientSecret` environment variable).

TIP

If you need more control over how the client secret is loaded, you can use

`GoogleIdpConfig(clientSecret: GoogleClientSecret.fromJsonString(...))` instead.

See the [customizations](#) page for details.

Create the endpoint

Create a new endpoint file in your server project (for example,

`my_project_server/lib/src/auth/google_idp_endpoint.dart`) alongside the existing auth endpoints. Extending the base class registers the sign-in methods with your server so your app can call them to complete the authentication flow:

```
import 'package:serverpod_auth_idp_server/providers/google.dart';

class GoogleIdpEndpoint extends GoogleIdpBaseEndpoint {}
```

Start the server

Start the server from your server project directory (for example, `my_project_server/`):

```
serverpod start
```

Then create and apply the migration for the provider's tables: in the `serverpod start` terminal, press **M** to create and apply the migration.

WARNING

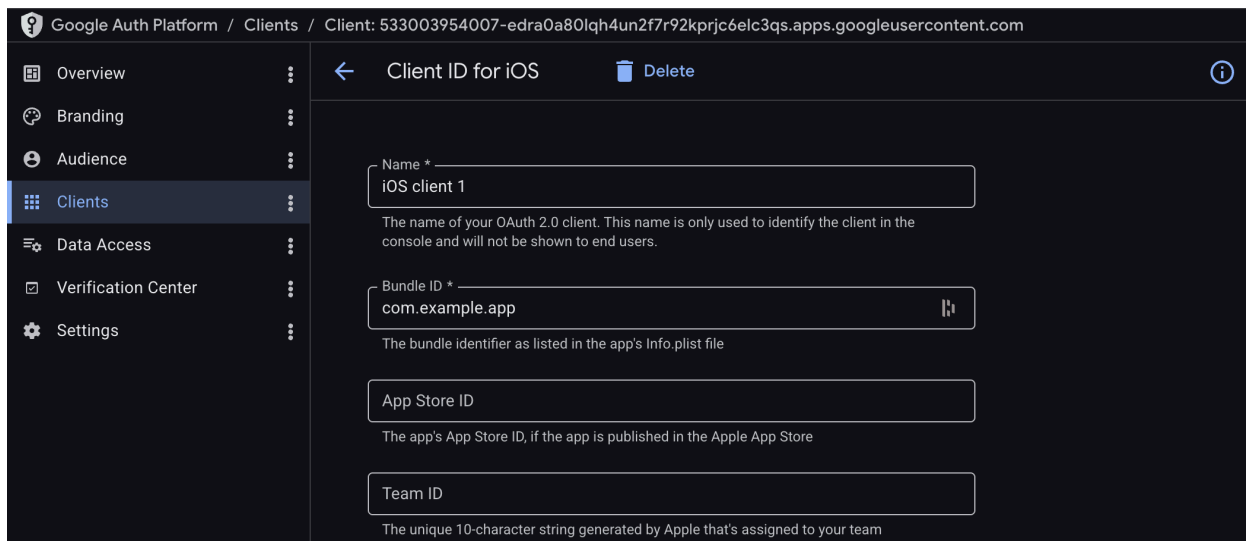
Skipping the migration will cause the server to crash at runtime when the Google provider tries to read or write user data. More detailed instructions can be found in the general [identity providers setup](#) section.

Client-side configuration

The Android and iOS integrations use the `google_sign_in` package under the hood, so any documentation there should also apply to this setup.

iOS

1. In the Google Auth Platform, navigate to **Clients** and click **Create Client**.
2. Select **iOS** as the application type.
3. Fill in your app's **Bundle ID** and any other required information.
4. Click **Create** and download the `.plist` file.



Google Auth Platform / Clients / Client: 533003954007-edra0a80lqh4un2f7r92kprjc6elc3qs.apps.googleusercontent.com

Overview
Branding
Audience
Clients
Data Access
Verification Center
Settings

Client ID for iOS Delete

Name *
iOS client 1
The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.

Bundle ID *
com.example.app
The bundle identifier as listed in the app's Info.plist file

App Store ID
The app's App Store ID, if the app is published in the Apple App Store

Team ID
The unique 10-character string generated by Apple that's assigned to your team

5. Open the `Info.plist` file in your Flutter project (for example, `my_project_flutter/ios/Runner/Info.plist`) and add the following keys inside the top-level `<dict>`:

```
<dict>
  ...
  <key>GIDClientID</key>
  <string>your_ios_client_id</string>
  <key>GIDServerClientID</key>
  <string>your_server_client_id</string>
</dict>
```

Replace `your_ios_client_id` with the `CLIENT_ID` value from the downloaded plist file, and `your_server_client_id` with the client ID from the [Web application OAuth client](#) you created earlier.

Add the URL scheme

To allow navigation back to the app after sign-in, add the URL scheme to the same `Info.plist` file. The scheme is the reversed client ID of your iOS app (found as `REVERSED_CLIENT_ID` in the downloaded plist file). Add the following inside the top-level `<dict>`:

```
<dict>
```

```
...  
<key>CFBundleURLTypes</key>  
<array>  
  <dict>  
    <key>CFBundleTypeRole</key>  
    <string>Editor</string>  
    <key>CFBundleURLSchemes</key>  
    <array>  
      <string>com.googleusercontent.apps.your_client_id</string>  
    </array>  
  </dict>  
</array>  
</dict>
```

Replace the URL scheme with your actual reversed client ID.

WARNING

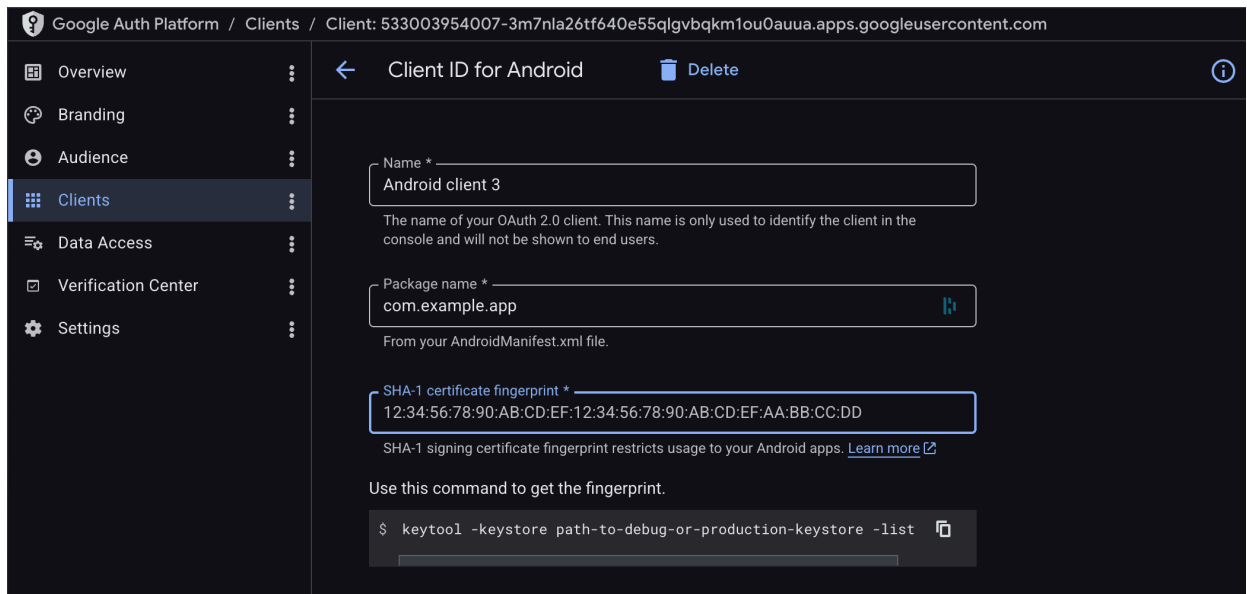
Without the URL scheme, the OAuth callback never returns to your app and sign-in silently hangs.

Android

1. In the Google Auth Platform, navigate to **Clients** and click **Create Client**.
2. Select **Android** as the application type.
3. Fill in your app's **Package name** and **SHA-1 certificate fingerprint**. You can get the debug SHA-1 hash by running this from your Flutter project's `android/` directory (for example, `my_project_flutter/android/`):

```
./gradlew signingReport
```

4. Click **Create**. This registers your app's package name and signing key with Google.



5. On Android, the sign-in SDK also needs to know your server's client ID. Pass the [Web application OAuth client's](#) ID as `serverClientId` when you initialize the client (covered in [Initialize the Google sign-in service](#) below). You can also pass it at build time with `--dart-define`. See [Configuring client IDs on the app](#).

NOTE

If your app uses Firebase (the `com.google.gms.google-services` Gradle plugin), you can skip step 5: the plugin supplies the server client ID from `google-services.json`. Re-download that file after creating the Web application client so it includes the web client entry.

WARNING

When testing against a local server, the Android emulator cannot reach `localhost`: sign-in completes, but the endpoint call fails with a connection error. See [troubleshooting](#) for pointing the app at your machine.

Web

On web, Google completes sign-in by redirecting the browser to a callback URL you control. This flow requires Serverpod to serve your Flutter web app on the **same origin** (same scheme, host, and port) as the callback route.

WARNING

The web flow only works from the **built** app served by Serverpod (`http://localhost:8082/` locally on default projects). Running the app with `flutter run -d chrome` fails, because Flutter's

dev server is a different origin than Serverpod and the browser blocks the sign-in callback. See [troubleshooting](#). For a hot-reload workflow, use the [separately-hosted Flutter web](#) flow instead.

To test locally, build your Flutter web app into Serverpod's `web/app/` directory and start the server:

```
flutter build web --base-href / --output ../my_project_server/web/app # from your
Flutter project
serverpod start --no-flutter # from your server
project
```

Replace `my_project_server` with your server package directory. Pass `--no-flutter` so `serverpod start` serves your prebuilt web app. Without the flag, it also runs a Flutter web dev server on a different port, which does not share Serverpod's origin.

Open `http://localhost:8082/` to test. Projects created with the website option serve the app under `/app` instead. Build those with `--base-href /app/` and open `/app`.

The examples below use port `8082` (Serverpod's default from `config/development.yaml`).

1. In your server project's `server.dart` file, before `pod.start()`, register the callback route:

```
import 'package:serverpod_auth_idp_server/core.dart';

// ...

pod.webServer.addRoute(
  FlutterWebAuth2CallbackRoute(),
  '/auth/callback',
);
```

The path `/auth/callback` can be anything, as long as it matches the URL you register with Google in step 2 and pass to `initializeGoogleSignIn` later.

In production, pass `host:` to restrict the route to your production domain:

```
pod.webServer.addRoute(
  FlutterWebAuth2CallbackRoute(host: 'my-awesome-project.serverpod.space'),
  '/auth/callback',
);
```

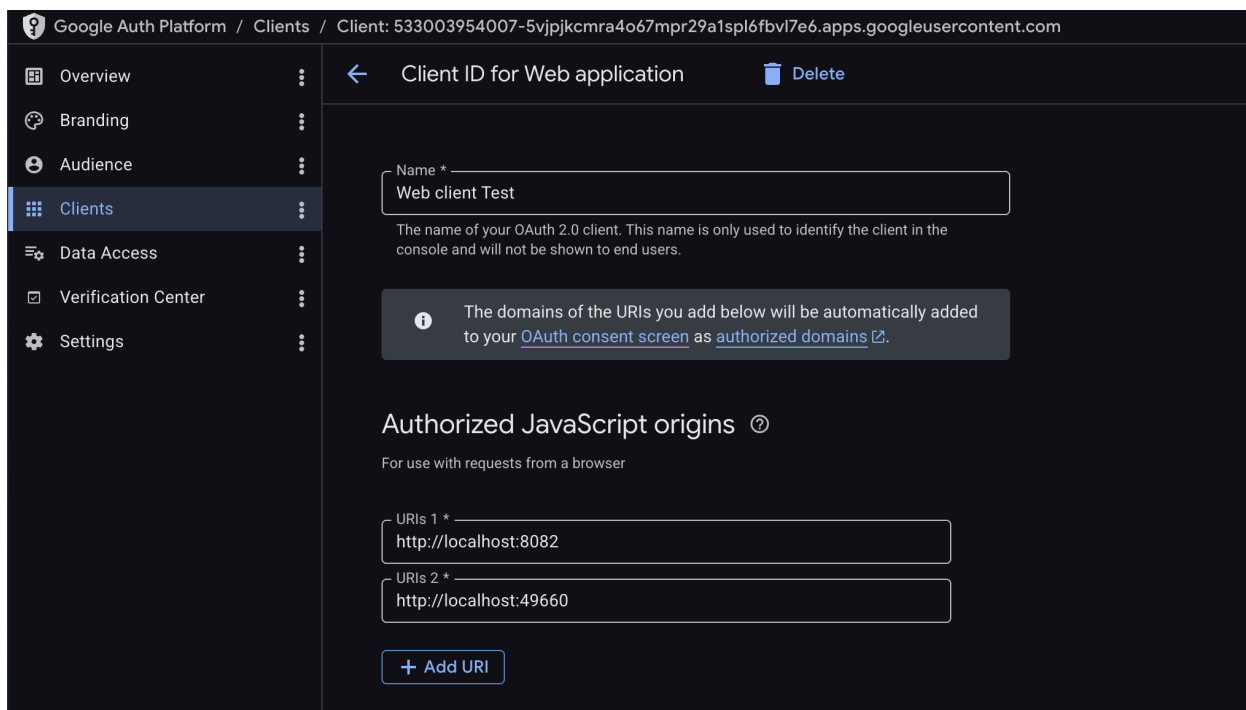
 **TIP**

The same route works for every OAuth2 PKCE provider. Register it once and reuse the callback URL for Google, GitHub, and any future provider.

2. Update the server OAuth client you created in the [previous section](#) and add the following:

- **Authorized JavaScript origins:** your Flutter web app's origin (for example, `http://localhost:8082` locally, or `https://my-awesome-project.serverpod.space` in production).
- **Authorized redirect URIs:** the full URL of the route you registered (for example, `http://localhost:8082/auth/callback` locally, or `https://my-awesome-project.serverpod.space/auth/callback` in production).

Add the same URL to `redirect_uris` in the `googleClientSecret` entry of `passwords.yaml` from [Store your credentials](#).



3. Pass the same URL to `initializeGoogleSignIn` via the `redirectUri` argument when you initialize the client (covered in [Initialize the Google sign-in service](#) below).

TIP

You can also pass the redirect URI via `--dart-define`. See [Configuring the web redirect URI](#) for the pattern.

Present the authentication UI

Initialize the Google sign-in service

In your Flutter app's `lib/client.dart`, the template creates the `Client` and calls `client.auth.initialize()` inside `initializeClient()`. Add `initializeGoogleSignIn()` right after that call:

```
unawaited(client.auth.initialize());
client.auth.initializeGoogleSignIn();
```

On web, the call needs `clientId` and `redirectUri` (matching the route from [Web](#)). On Android, it needs `serverClientId` from [Android](#). On iOS, the SDK reads its config from `Info.plist`, and passing `serverClientId` is fine since it holds the same value as `GIDServerClientID`:

```
if (kIsWeb) {
  client.auth.initializeGoogleSignIn(
    clientId: '<web_client_id>.apps.googleusercontent.com',
    redirectUri: 'http://localhost:8082/auth/callback',
  );
} else {
  client.auth.initializeGoogleSignIn(
    serverClientId: '<web_client_id>.apps.googleusercontent.com',
  );
}
```

Swap the redirect URI for your production URL when deploying. See [Configuring the web redirect URI](#) to avoid hard-coding it per environment.

⚠ WARNING

On web, the app Serverpod serves is the build you created in [Web setup](#). After changing `client.dart` (for example the `redirectUri`), run the build command again and hard-reload the browser. A stale build keeps sending the old values, and sign-in fails with `redirect_uri_mismatch`.

Show the Google sign-in button

New projects include a `SignInScreen` widget at `lib/screens/sign_in_screen.dart`. The version below is trimmed to the essentials. It listens to `client.auth.authInfoListenable` and swaps between `SignInWidget` while the user is signed out and the `child` you pass it once they sign in. The `SignInWidget` auto-detects which identity provider endpoints are registered on the server, so once `GoogleIdpEndpoint` is exposed and the client code has been regenerated, the Google button appears inside it.

```
import 'package:flutter/material.dart';
import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';
```

```

import '../client.dart';

class SignInScreen extends StatefulWidget {
  final Widget child;
  const SignInScreen({super.key, required this.child});

  @override
  State<SignInScreen> createState() => _SignInScreenState();
}

class _SignInScreenState extends State<SignInScreen> {
  bool _isSignedIn = false;

  @override
  void initState() {
    super.initState();
    client.auth.authInfoListenable.addListener(_updateSignedInState);
    _isSignedIn = client.auth.isAuthenticated;
  }

  @override
  void dispose() {
    client.auth.authInfoListenable.removeListener(_updateSignedInState);
    super.dispose();
  }

  void _updateSignedInState() {
    setState(() {
      _isSignedIn = client.auth.isAuthenticated;
    });
  }

  @override
  Widget build(BuildContext context) {
    return _isSignedIn
      ? widget.child
      : Center(
        child: SignInWidget(
          client: client,
          onError: (error) {
            ScaffoldMessenger.of(context).showSnackBar(
              SnackBar(content: Text('Authentication failed: $error')),
            );
          },
        ),
      );
  }
}

```

⚠ WARNING

The `SignInScreen` listener is what swaps to your authenticated UI. If you also pass an `onAuthenticated` callback to `SignInWidget`, use it for transient feedback only (snackbars,

analytics). Driving navigation from `onAuthenticated` instead of the listener sends the user back to sign-in on every app restart even though their session is still valid.

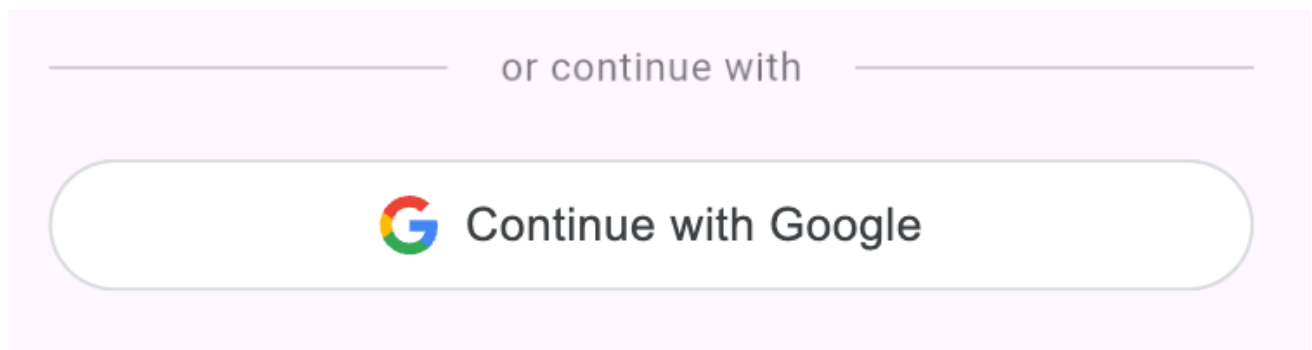
In `main.dart`, the template wires this into `MyHomePage.build()`'s `Scaffold` behind a commented block. Comment out `body: const GreetingsScreen()` and uncomment the `SignInScreen(...)` block beneath it:

```
body: SignInScreen(  
  child: GreetingsScreen(  
    onSignOut: () async {  
      await client.auth.signOutDevice();  
    },  
  ),  
),
```

The running app does not include your `initializeGoogleSignIn` call until you rebuild or restart it:

- **Web:** Serverpod serves the built app, so run the `flutter build web` command from [Web](#) again, then hard-reload the browser. Until then, the Google button stays hidden.
- **Android and iOS:** hot reload does not re-run `initializeClient()`, so hot restart the app by pressing **R** in the `serverpod start` terminal, or rerun `flutter run`. Before you restart, Android sign-in fails with `serverClientId must be provided`.

The `SignInWidget` renders the standard Google sign-in button:



To change the button's theme or build a fully custom UI, see [Customizations](#).

TIP

If you run into issues, see the [troubleshooting guide](#).

Publishing to production

Before going live, complete the following steps:

1. Verify your authorized domain

Google's **Authorized domains** field on the [Branding](#) page accepts only the **top private domain** (the root). Once the root is verified, every subdomain under it is automatically authorized, and you do not need to add each project subdomain separately.

If you deploy on Serverpod Cloud under a `*.serverpod.space` subdomain, `serverpod.space` is already verified by Serverpod. Add `serverpod.space` to **Authorized domains** in the Google Auth Platform; no DNS verification is required on your end.

For a custom domain, verify ownership of your root domain (for example, `example.com`) at [Google Search Console](#) by adding the DNS TXT record Google provides. After verification completes, add the root to **Authorized domains** in the Google Auth Platform.

TIP

A single verified root authorizes all of its subdomains. If Google rejects a domain you add, you are likely entering a full subdomain instead of the root.

2. Update the OAuth redirect URIs

Go back to the [server OAuth client](#) in the Google Auth Platform and add your production URLs:

- **Authorized JavaScript origins:** your production Flutter web app's origin (for example, `https://my-awesome-project.serverpod.space`).
- **Authorized redirect URIs:** the production callback URL. From the standard [Web setup](#), this is the route URL you registered (for example, `https://my-awesome-project.serverpod.space/auth/callback`). For the [separately-hosted Flutter web](#) fallback, it's the full URL where `auth.html` is served on your production Flutter web host (for example, `https://app.example.com/auth.html`).

Replace the URLs with your actual production address. On Serverpod Cloud, your project is served from `https://<project-id>.serverpod.space`.

3. Set production credentials

Production runs out of the `production:` section of `passwords.yaml`, which is separate from the `development:` section you populated during setup. Adding production credentials does not replace your development ones, both stay in place and Serverpod picks the right set based on the run mode.

The production `googleClientSecret` reuses the same web client ID and secret from setup, but lists your production redirect URI rather than the development one. If you use a different OAuth client for production, [create another web client](#) first and use its values below.

Pick the path that matches your deployment:

Self-hosted

Add `googleClientSecret` to the `production:` section of `passwords.yaml` with the production redirect URI:

```
production:
  # ... existing keys ...
  googleClientSecret: |
    {
      "web": {
        "client_id": "<web_client_id>.apps.googleusercontent.com",
        "client_secret": "your-client-secret",
        "redirect_uris": ["https://my-awesome-project.serverpod.space/auth/callback"]
      }
    }
```

Alternatively, set the `SERVERPOD_PASSWORD_googleClientSecret` environment variable on your production server with the same JSON value.

Serverpod Cloud

Use `https://<project-id>.serverpod.space/auth/callback` as the redirect URI in the JSON (matching the route you registered in [Web setup](#)). Save it to a file and use `serverpod cloud password set` with `--from-file`:

```
serverpod cloud password set googleClientSecret --from-file path/to/google-client-secret.json
```

Run this from your linked server project directory, or pass `--project <project-id>` on the call. See the [Serverpod Cloud passwords guide](#) for project linking and other options.

4. Update the Android OAuth client with the release SHA-1

The Android OAuth client you created during setup uses your debug SHA-1 fingerprint. Release builds are signed with a different key, so you need to add the release SHA-1 as well.

If you use Google Play App Signing (the default for new apps), get the SHA-1 from the Play Console: **Setup > App integrity > App signing key certificate**. Make sure to use the **app signing key** SHA-1, not the upload key SHA-1.

If you manage your own release keystore, get the SHA-1 from it directly:

```
keytool -list -v -keystore your-release-key.jks -alias your-key-alias
```

Once you have the SHA-1, go back to your Android OAuth client in the Google Auth Platform and add it under **SHA-1 certificate fingerprint**.

 WARNING

Forgetting this step is one of the most common reasons Google sign-in works in debug builds but silently fails after publishing to the Play Store.

5. Publish the OAuth consent screen

While the app is in **Testing** mode, only the test users you added on the [Audience](#) page, in the Google Auth Platform, can sign in. All other users will see an error.

Navigate to the **Audience** page and click **Publish App** to allow any Google account to sign in. If your app uses sensitive or restricted scopes, Google may require a verification review before publishing.